

On arbitrary binary trees of hypercubes

Shin-Feng Tsai

Department of Information Management
Southern Taiwan University of
Technology
n9090029@webmail.stut.edu.tw

Hui-Ling Huang

Department of Information Management
Southern Taiwan University of
Technology
hlhuang@mail.stut.edu.tw

摘要

嵌入樹 (tree) 於超立方體 (hypercube) 的問題在過去二十年中一直引起許多研究者的關注。超立方體是一個最受注意和討論的連結網路結構，而且被實際建構成為真正可用的平行電腦，這是因為它有著相當好的性質，如模組化、對稱性(symmetry)、規則性(regularity)、較小的直徑(diameter)和容錯(fault-tolerant)能力等等。而樹更是重要的結構，在二元樹上已被研究出很多相當有效率的演算法，可以解決各種應用問題。為了將現存某一網路上的演算法能夠輕易的轉移使用在另一網路上，嵌入(embedding)的問題就因應而生。嵌入在多處理機連結網路上是一重要的研究主題，假如我們能將某一網路嵌入另一網路，即代表在此網路上所設計的演算法都可很容易地轉移至另一網路上。本篇論文改進之前Chen與Stallmann的研究成果，我們提出一個新的遞迴演算法將任意二元樹嵌入超立方體結構中，在同樣的條件之下我們的嵌入演算法其膨脹(dilation)係數必定小於等於Chen與Stallmann的方法。

關鍵詞： 樹(tree)、超立方體(hypercube)、嵌入(embedding)、前序追蹤(preorder traversal)、格雷碼(Gray code)、膨脹係數(dilation)

一、簡介

近幾年來由於 VLSI 技術的進步，有許多使用多處理機的連結網路被提出來用以解決複雜的平行計算，如 NCUBE/ten、Cosmic Cube 和 Intel iPSC 系列等超立方體(hypercube)結構是較具代表性且已製造出來作為商業用途的多處理機系統[12, 13, 19, 20]，這些系統甚至可連結數千或數百個多處理器。由於超立方體有許多相當好的性質，因此在超立方體上已有相當多的研究成果，如廣播(broadcasting)、排序(sorting)、工作配置(job

allocation)、排程(scheduling)和其他各種應用問題[6, 7, 18, 23]。

由於近年來在其他各類的連結網路上，已設計出許多的平行演算法去解各式各樣的問題。為了將現存在某一網路上的演算法能夠輕易的轉移使用在另一網路上，嵌入(embedding)的問題就因應而生。我們將連結網路視為一種圖形結構，將處理器視為節點(node)，處理器和處理器之間的連線視為邊(edge)。在圖論中將客圖(guest graph)嵌入主圖(host graph)意指在主圖中找出一個子圖與客圖同構(isomorphic)。圖形嵌入在多處理機連結網路上是一重要的研究主題。

在樹結構上有著許多相當有效率的演算法，如排序及 divide and conquer 演算法等，尤其在資料庫上的一些演算法更可以在以樹為主的資料結構上得到相當好的執行效率。因此如何將樹結構嵌入超立方體中，使得這些在樹結構上相當有效率的演算法可以很容易地轉移至實際的超立方體平行電腦上是一個相當重要的問題。

近年來對於如何將各種樹結構嵌入超立方體已有很多研究成果，包括二元樹[2, 10, 14]、X-tree[9]、tree machines [1]、網狀樹[11, 16]、延伸樹及平衡延伸樹[13]等等。其中又以二元樹有最多研究成果，例如完全二元樹[15]、不完全二元樹[22]、任意二元樹[3-5, 21, 24]、full-ringed二元樹[15]等。

本篇論文主要是改進之前Chen與Stallmann的研究成果[8]，我們提出一個新的演算法將任意二元樹嵌入超立方體結構中，其膨脹(dilation)係數必定小於等於Chen與Stallmann的方法，並且擴張係數為1、平均邊膨脹係數不超過2。

本篇論文其餘各節的內容如下：第二節說明符號與定義，在第三節中我們將說明嵌入任意二元樹於超立方體中的演算法，並在第四節做總結。

二、符號與定義

首先我們定義超立方體結構， n 維超立方體 Q_n 有 2^n 個節點，每個節點以 n 個位元表示。任

兩個節點 u 和 v ，若其節點表示式只有一個位元不同，則此兩點之間有一個邊(edge)相連，我們以 $\{u, v\}$ 表示兩節點的邊。

我們定義任意二元樹 T_n 有 2^n 個節點，樹 T_n 的根節點為 $r(T_n)$ 。任一節點 v 至多有兩個子樹分別是 $L(v)$ 和 $R(v)$ ，在不失一般性的前提下，我們將較少節點數的子樹視作左子樹 $L(v)$ ，另一個子樹則為右子樹 $R(v)$ 。 $|T_n|$ 代表樹 T_n 的節點數。

我們定義任意兩個節點 u 和 v 其 hamming distance 為 $d(u, v)$ 。我們的嵌入方法將運用到格雷碼(Gray code)，任意連續兩個格雷碼的 hamming distance 為 1。我們定義 k 位元的格雷碼序列為 G_k ，它可由以下的遞迴函數建構而成：

$$\begin{cases} G_1 = (0, 1) \\ G_k = (0G_{k-1}, 1G_{k-1}^R) \end{cases}$$

其中 G_{k-1}^R 為 G_{k-1} 的逆向運算子。

圖一所示為 4 個位元所形成的 Gray code G_4 。我們定義其中第 i 個 Gray code 為 $\text{graycode}(i)$ 。所以圖一中 $\text{graycode}(3) = 0011$ ， $\text{graycode}(7) = 0101$ 。

NUMBER	GRAY CODE
1	0000
2	0001
3	0011
4	0010
5	0110
6	0111
7	0101
8	0100
9	1100
10	1101
11	1111
12	1110
13	1010
14	1011
15	1001
16	1000

圖一 . 4 位元的格雷碼 G_4

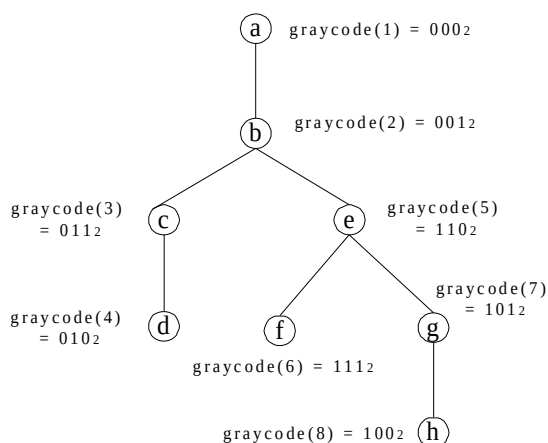
在嵌入的過程中，通常我們使用 4 個成本參數來評估嵌入的結果[15, 17]，它們分別是負載(load)、擴張(expansion)、膨脹(dilation)和擁塞

(congestion)。我們定義客圖 G 嵌入主圖 H 的負載是最多 G 上的節點映射至 H 上的單一節點之數量。擴張是 H 的節點數目除以 G 的節點數目。邊膨脹是 G 的一個邊映射至 H 上的路徑之長度，我們定義整個嵌入的膨脹值為 G 所有的邊膨脹值取最大值。邊擁塞則是 G 上的邊共用 H 上的單一邊之數量，整個嵌入的擁塞值是所有的邊擁塞值取最大值。負載可用以衡量工作的處理時間(the processing time of tasks)，擴張可用以衡量處理機的有效使用率(processor utilization)，邊膨脹可用以測量通訊的效率(communication performance)，而邊擁塞可用以測量訊息的等候延遲(the queueing delay of messages)。一個好的嵌入結果，是需要這 4 個參數能盡可能的小，但是在這 4 個參數之中，有些彼此間有相互消長(tradeoff)的關係，故在本篇論文中，我們固定住負載及擴張值等於 1，來降低平均膨脹係數。

三、嵌入二元樹演算法

之前 Chen 與 Stallmann 提出一個嵌入任意二元樹於超立方體的先序格雷碼演算法(Preorder-Gray-code Algorithm) [8]。此演算法固定由較小的左子樹開始追蹤，依照節點追蹤的順序給予格雷碼，圖二為應用此先序演算法將二元樹 T_3 嵌入三維的超立方體 Q_3 的結果。此演算法的追蹤對映有一個特點，就是比較小的子樹都會先被追蹤，我們再將此追蹤的順序編號轉成格雷碼並對映到超立方體上的節點。因為格雷碼具有相鄰兩碼其漢明距離(Hamming distance)為 1 的特性，因此任一節點與其左兒子節點的邊膨脹均為 1，但追蹤較右子樹的時候通常會造成較大的邊膨脹，如圖二的邊 $\{b, e\}$ 其邊膨脹係數是 $d(\text{graycode}(2), \text{graycode}(5)) = 3$ 。而邊 $\{e, g\}$ 的邊膨脹係數是 $d(\text{graycode}(2), \text{graycode}(5)) = 2$ 。

由於觀察到這個現象，因此我們提出一個新的演算法希望能降低原始前序嵌入法所造成右子樹膨脹係數過大的問題。當追蹤到根節點的右子樹的時候並不是直接用前序的方法，而是先比較前序追蹤與另一種中序追蹤，對於根節點與其右子樹的邊(edge)所造成的邊膨脹係數，再採用較小邊膨脹的追蹤方法，因此每回在追蹤右子樹時有可能採用前序追蹤，也有可能改為中序追蹤，但我們若採用中序追蹤，雖然可以改善根節與其右子樹的邊膨脹，但其右子樹內部的邊膨脹值可能反而會增加，為了要維持右子樹內部任一節點與其左兒子節點的邊膨脹係數保持為 1，我們採用反向前序追蹤的技巧如此將減少根節點與其右子樹這個邊的邊膨脹。



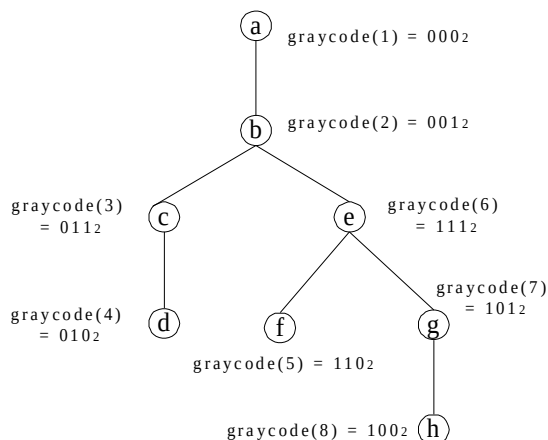
圖二 . Preorder-Gray-code 嵌入演算法

附錄為我們所提出的演算法，它增進強化了[8]的研究結果。此演算法 $PIG(T, i, j)$ 是將 $graycode(i)$ 到 $graycode(j)$ 依序指派到樹 T 中，且此樹所有節點的數目為 $j-i+1, i \leq j$ 。例如當二元樹 T_3 嵌入到超立方體 Q_3 時我們呼叫 $PIG(T_3, 1, 8)$ 。此演算法以根節點 v 為中心，首先追蹤根節點，之後比較其兩個子樹節點數目的多寡，令較小子樹為 $L(v)$ ，再先向左子樹 $L(v)$ 追蹤。我們演算法最主要的不同點在於追蹤右子樹的部份，在追蹤右子樹時，我們會比較直接用前序追蹤或是採用中序追蹤兩種不同的追蹤法所得到的膨脹係數，我們在追蹤右子樹時將採用比較過後膨脹係數較低的追蹤法。如此可以保證根節點與其左子樹的膨脹係數為 1，而根節點與其右子樹的膨脹係數必定小於等於原始[8]演算法所得的值。

中序追蹤法 $Inorder(T, p, q)$ 將 $graycode(p)$ 到 $graycode(q)$ 依序指派到樹 T 中， $p \leq q$ 。但它並不是單純的中序追蹤，我們令 v 為 T 的根節點，即 $v = r(T)$ 。首先呼叫 $Inverse$ 程序去追蹤根節點 v 的左子樹，將 $graycode(p)$ 到 $graycode(p+|L(v)|-1)$ 指派給 $L(v)$ ，之後追蹤根節點 v ，將 $graycode(p+|L(v)|)$ 指派給節點 v ，最後呼叫 PIG 程序追蹤 v 的右子樹，將 $graycode(p+|L(v)|+1)$ 到 $graycode(q)$ 指派給 $R(v)$ 。而 $Inverse(T, y, x)$ 將 $graycode(y)$ 到 $graycode(x)$ 依序指派到樹 T 中， $y \geq x$ 。它是一種反向追蹤的程序，我們也可以把它看成是一種倒著順序的前序追蹤，即編號由最大一個編號開始，由子樹的根節點向下遞減指派，如圖三我們呼叫 $Inorder(R(b), 5, 8)$ ，則 $f(e) = graycode(6)$ ，之後呼叫 $Inverse(L(e), 5, 5)$ 則 $f(f) = graycode(5)$ 。 $graycode(M)$ 為一個函數，它傳回第 M 個格雷碼。此論文的演算法重點是 $Inorder(T, p, q)$ 追蹤程序，如果用 chen[8] 的方法， $\{b, e\}$ 的邊膨脹係數值是 3， $\{e, g\}$ 的邊膨脹係數是 2，而執行我們的

嵌入演算法將使膨脹係數降為 2，且 $\{e, g\}$ 的邊膨脹係數減為 1。因此，執行我們的演算法不但減少 $\{b, e\}$ 的邊膨脹也同時減少 $\{e, g\}$ 的邊膨脹，而讓平均膨脹係數更小。

但若是採用 Inorder 及反向前序追蹤法，雖然可以保證此根節點與其右子樹的膨脹係數小於原始 Preorder-Gray-code 演算法的膨脹係數，但針對 $R(v)$ 後續的追蹤卻有可能發生其膨脹係數值較原始演算法為大的情形；即我們改善了較上階層節點對其右子樹的膨脹係數，但卻可能造成在較下階層節點與其右子樹節點的膨脹係數值變差。因此為了避免這樣的情況發生，我們必須遞迴地檢視比較嵌入右子樹的部份，在採反向前序追蹤程序 $Inverse(T, y, x)$ 時，追蹤右子樹的部份會再次比較反向前序追蹤或中序追蹤，相同的我們將採用膨脹係數較低的追蹤法，一旦採用中序追蹤法時將呼叫 $RInorder$ 程序，它與 Inorder 程序類似但因在反向追蹤程序中參數呼叫的方向性不同，因此無法直接用 Inorder 程序，遞迴地檢視比較右子樹嵌入將可改善進入反向前序追蹤時可能造成右子樹膨脹係數比原前序追蹤還大的情形。



圖三 . 利用我們的演算法將 T_3 嵌入到 Q_3

定理 1. 利用我們的演算法嵌入樹於超立方體其擴張係數為 1 且平均邊膨脹係數不會超過 2。

證明：令 T 為以 v 為根節點的二元樹， $|T| = n, l = |L(v)|, r = |R(v)|$ ，所以 $n = l + r + 1$ 。由格雷碼的特性[3]我們知道邊 $\{v, L(v)\}$ 邊膨脹永遠是 1，而邊 $\{v, R(v)\}$ 的邊膨脹會是小於或等於 $\lceil \log(l+1)+1 \rceil$ 。在先序中因為較小子樹總是先標

記，所以我們假設 $l \leq (n-1)/2$ 和 $r \geq (n-1)/2$ 。

我們定義 $D(n)$ 代表 n 個節點的樹 T 嵌入超立方體的所有邊膨脹係數總和，則

$$\begin{cases} D(1) = 0, \\ D(n) \leq D(r) + 1, \text{ if } l = 0 \\ D(n) \leq D(l) + D(r) + 1 + \lceil \log(l+1) + 1 \rceil, \text{ if } l \neq 0 \end{cases}$$

$$\begin{aligned} D(n) &\leq D(l) + D(r) + 1 + (\lceil \log(l+1) \rceil + 1) \\ &\leq 2l - \lceil \log(l+1) \rceil - 1 + 2r - \lceil \log(r+1) \rceil \\ &\quad - 1 + \lceil \log(l+1) \rceil + 2 \\ &= 2(l+r) - \lceil \log(r+1) \rceil \\ &\leq 2n - \left\lceil \log\left(\frac{n-1}{2} + 1\right) \right\rceil - 2 \\ &= 2n - \left\lceil \log\left(\frac{n+1}{2}\right) \right\rceil - 2 \\ &= 2n - \lceil \log(n+1) \rceil - 1 \end{aligned}$$

$$\text{則平均膨脹係數為 } \leq \left(\frac{2n - \lceil \log(n+1) - 1 \rceil}{n-1} \right) = 2$$

Q. E. D.

此平均膨脹係數的上限值雖然與[8]所提出的演算法相同，但因為我們的演算法額外改善每一倍節點與其右子樹的邊膨脹係數值，因此一般所得到的平均膨脹係數會遠比上限值小。

四、總結

針對樹狀結構已有很多演算法去解決各式各樣的問題。如何將樹狀結構嵌入超立方體中，使得這些在樹結構上相當有效率的演算法可以很容易地轉移至實際的超立方體平行電腦上是一個相當重要的問題。

本篇論文針對任意二元樹 T_n 嵌入到超方體 Q_n 提出了一個演算法，由 T_n 嵌入到 Q_n 在固定擴張及負載係數為 1 的情況下獲得比[8]更好的膨脹係數值。我們希望未來可以找到一個最佳膨脹係數的嵌入演算法。雖然目前我們無法證明此演算法為最佳的嵌入演算法，但我們相信此演算法已大幅降低膨脹係數，並且有可能再加以改善而得

到最佳的嵌入演算法，這將是我們未來的首要研究目標。

五、參考文獻

- [1] J. L. Bentley, and H. T. Kung, "A tree machine for searching problems," *Proceedings of International Conference on Parallel Processing*, pp. 257-266, 1979.
- [2] S. N. Bhatt, and I. C. F. Ipsen, "How to embed trees in hypercubes," *Yale University Res. Rep.*, RR-443, Dec. 1985.
- [3] S. N. Bhatt, F. R. K. Chung, F. T. Leighton, and A. L. Rosenberg, "Efficient embedding of trees in hypercubes," *SIAM J. Comput.*, vol. 21, pp. 151-162, Feb. 1992.
- [4] S. N. Bhatt, F. Chung, T. Leighton, and Rosenberg, "A. Optimal simulation of tree machines," *Proceedings of the 1986 27th Annual Symposium of Foundations of Computer Science*, pp. 274-282.
- [5] L. Bhuyan, and D. P. Agrawal, "Generalized hypercubes and hyperbus structures for a computer network," *IEEE Transactions on Computers*, C-33, pp. 323-333, 1984.
- [6] J. Blazewicz, and M. Drozdowski, "Scheduling divisible jobs on hypercubes," *Parallel Computing*, vol. 21, pp. 1945-1956, 1995.
- [7] Y. W. Chen, and K. L. Chung, "Improved fault-tolerant sorting algorithm in hypercubes," *Theoretical Computer Science*, vol. 255, pp. 649-658, 2001.
- [8] W. Chen, and M. F. M. Stallmann, "On Embedding Binary Trees into Hypercubes," *Journal of Parallel and Distributed Computing*, vol. 24, pp. 132-138, 1995.
- [9] A. M. Despain, and D. A. Patterson, "X-tree: A tree structured multiprocessor computer architecture," *Proceedings of the 5th Annual Symposium on Computer Architecture*, pp. 144-151, 1978.
- [10] S. R. Desphance, and R. Jenevein, "Scalability of binary trees on a hypercube," *Proceedings of International Conference on Parallel Processing*, pp. 661-668, 1986.
- [11] K. Efe, "Embedding mesh of trees in the hypercube," *Journal of Parallel and Distributed Computing*, vol. 11, pp. 222-230, 1991.
- [12] J. P. Hayes, T. N. Mudge, Q. F. Stout, S. Colley and J. Palmer, "A Microprocessor-based hypercube supercomputer," *IEEE Micro*,

- vol. 6, pp. 6-17, 1986.
- [13] C. T. Ho, and S. L. Johnsson, "Spanning balanced trees in Boolean cubes," *SIAM J. Sci. Statist. Comput.*, vol.10-4, pp. 607-630, July 1989.
- [14] E. Horowitz, and A. Zorat, "The binary tree as an interconnection network : Applications to multiprocessor systems and VLSI," *IEEE Transactions on Computers*, vol. 30, pp. 247-253, 1981.
- [15] F. T. Leighton, *Introduction to Parallel Algorithms and Architectures: Arrays, Trees and Hypercubes*, Morgan Kaufmann, San Mateo CA, 1992.
- [16] F. T. Leighton, "Parallel computing using mesh of trees," *Proceedings of the Workshop on Graph-Theoretic Concepts in Computer Science*, pp. 200-218, 1983.
- [17] I. L. Leiss, and H. N. Reddy, "Embedding complete binary trees into hypercubes," *Inform. Process. Lett.*, vol. 38, pp. 197-199, May 1991.
- [18] P. Mohapatra, "Dynamic real-time task scheduling on hypercubes," *Journal of Parallel and Distributed Computing*, vol. 46, pp. 91-100, 1997.
- [19] D. A. Read and R. M. Fujimoto, *Multi-computer Network: Message-Based Parallel Processing*, MIT Press, Cambridge MA, 1987.
- [20] Y. Saad and M. H. Schultz, "Topological properties of hypercubes," *IEEE Transactions on Computers*, vol. C-37, pp. 867-872, 1988.
- [21] A. Wagner, "Embedding arbitrary binary trees in a hypercube," *Journal of Parallel and Distributed Computing*, vol. 7, pp. 503-520, 1989.
- [22] A. Y. Wu, "Embedding of tree networks into hypercubes," *Journal of Parallel and Distributed Computing*, vol. 2, pp. 238-249, 1985.
- [23] J. Wu, "Optimal broadcasting in hypercubes with link faults using limited global information," *Journal of Systems Architecture*, vol. 42, pp. 367-380, 1996.
- [24] J. Wu, E. B. Fernandez, and Y. Luo, "Embedding of Binomial Trees in Hypercubes with Link Faults," *Journal of Parallel and Distributed Computing*, vol. 54, pp. 59-74, 1998.

附錄：

Procedure PIG(T, i, j)

```

/* 將 graycode(i)到 graycode(j)指派給樹 T,
i≤j */
{
    v = r(T);
    f(v) = graycode(i);
    /* 將 graycode(i)指派給節點 v */
    if i = j then return
    else
        i = i + 1

    Let the children of v be L(v) and R(v) with
        |L(v)| ≤ |R(v)|;
    if |L(v)| = 0 then L(v) = R(v);
    PIG(L(v), i, i + |L(v)| - 1);
    if d(f(v), graycode(|L(v)| + 2)) ≤
        d(f(v), graycode(|L(v)| + |L(R(v))| + 2))
    then PIG(R(v), i + |L(v)|, j);
    else
        j = i + |L(v)| + |L(R(v))|;
        f(v) = graycode(j);
        j = j - 1;
        Inorder(R(v), i + |L(v)|, j);
}

Procedure Inorder( $T, p, q$ ) /*  $p \leq q$  */
{
    v = r(T);
    i = p + |L(v)|;
    Inverse(L(v), i - 1, p);
    f(v) = graycode(i);
    PIG(R(v), i + 1, q);
}

Procedure Inverse( $T, y, x$ ) /*  $x \leq y$  */
{
    v = r(T);
    f(v) = graycode(y);
    if y = x then return
    else
        y = y - 1

    Let the children of v be L(v) and R(v) with
        |L(v)| ≤ |R(v)|;
    if |L(v)| = 0 then L(v) = R(v);
    Inverse(L(v), y, y - |L(v)| + 1);
    if d(f(v), graycode(|y - |L(v)||)) ≤
        d(f(v), graycode(|y - |L(v)| + |L(R(v))|))
    then Inverse(R(v), y - |L(v)|, x);
    else
        x = y - |L(v)| - |L(R(v))|;
        f(v) = graycode(x);
        x = x + 1;
        RInorder(R(v), x, y - |L(v)|);
}

```

```

Procedure RInorder ( $T, a, b$ ) /*  $a \leq b$  */
{
   $v = r(T)$ ;
   $i = b - |L(v)|$ ;
  Inverse( $R(v), i+1, a$ );
   $f(v) = \text{graycode}(i)$ ;
  PIG( $L(v), i+1, i+|L(v)|$ );
}

```